



TOBB ETÜ
Ekonomi ve Teknoloji Üniversitesi

TOBB EKONOMİ VE TEKNOLOJİ ÜNİVERSİTESİ

Bilgisayar / Yapay Zekâ Mühendisliği

BİL 495 — YAP 495

Analysis Report

Analiz Raporu

Sistemin modeli: kullanıcı senaryoları, use-case, sınıf modeli, dinamik modeller, UI

Proje Adı	AgentLens
Takım Üyeleri	Anıl Özişler, İrem Özdemir, Yiğit Yıldız, Arda Günaydın, Yusuf Mert Usta
Danışman	Shadi Bikas
Akademik Dönem	2025-2026 Yaz Dönemi
Teslim Tarihi	12.07.2026

1.Introduction.....	
2.Proposed system.....	
2.1 Overview.....	
2.2 Functional Requirements.....	
2.2.1 Veri Toplama ve Saklama.....	
2.2.2 Bağlam Oluşturma.....	
2.2.3 Tespit ve Analiz.....	
2.2.4 Raporlama ve Görselleştirme.....	
2.2.5 Veri Yönetimi ve Değerlendirme.....	
2.2.6 Gizlilik ve Güvenlik.....	
2.3 Nonfunctional Requirements.....	
2.3.1 Performans.....	
2.3.2 Doğruluk ve Kalite.....	
2.3.3 Modülerlik ve Bakım.....	
2.3.4 Taşınabilirlik ve Entegrasyon.....	
2.3.5 Güvenlik ve Gizlilik.....	
2.3.6 Güvenilirlik.....	
2.3.7 Verimlilik ve Çevresel Etki.....	
2.3.8 Kullanılabilirlik.....	
2.4 Pseudo requirements.....	
3. System models.....	
3.1 Scenarios.....	
3.1.1 Mutlu Yol Senaryoları (Happy Paths).....	
Senaryo 1: Gerçek Zamanlı Sorunsuz Konuşma Akışı.....	
Senaryo 2: Gerçek Zamanlı Hata Tespiti ve Raporlanması.....	
Senaryo 3: Çevrimdışı (Offline) Log Analizi.....	
Senaryo 4: Sentetik Veri ile Performans Değerlendirmesi (Yeni).....	
3.1.2 Hata Yolu Senaryoları (Error Paths).....	
Hata Yolu 1: Zorunlu Alanı Eksik Log Kaydının İşlenmesi.....	
Hata Yolu 2: Formatlama Hatası ve Yeniden İşleme (Yeni).....	
3.2 Use Case Model.....	
3.3 Object and Class Model.....	
3.4 Dynamic Models.....	
3.5 User Interface — navigation & mock-ups.....	
4. Glossary.....	
References.....	

1.Introduction

Bu raporun amacı, çok ajanlı dil modeli sistemlerinde (MAS) iletişim ve koordinasyon kaynaklı başarısızlıkları otomatik olarak tespit edip açıklamak üzere tasarlanan AgentLens projesinin sistem mimarisini, yapısal gereksinimlerini ve dinamik davranış modellerini kapsamlı bir şekilde göstermektir. Geliştirme aşamasına geçilmeden önce hazırlanan bu analiz belgesi; sistemin karşılaması gereken fonksiyonel, fonksiyonel olmayan ve teknolojik kısıt (pseudo) gereksinimlerini net bir şekilde tanımlarken, kullanım senaryoları ve sistem bileşenleri arasındaki etkileşimleri gösteren dinamik modeller aracılığıyla geliştirme süreci için temel bir rehber oluşturur. Bu doküman, Bu rapor, projeyi değerlendirecek olan akademik kadromuz ve danışmanımız başta olmak üzere, sistemin geliştirilmesinden sorumlu ekip üyelerimiz için temel bir referans kılavuzu olarak hazırlanmıştır. Ayrıca, ileride AgentLens'i kendi çok ajanlı sistemlerine entegre etmek veya dışarıdan sağlanan log kayıtları üzerinden çevrimdışı hata analizi yapmak isteyen araştırmacılar ve geliştiriciler de bu dokümanın hedeflenen okuyucu kitlesi içindedir.

2.Proposed system

Bu bölüm AgentLens sisteminin önerilen yapısını tanımlar. Önce sistemin genel işleyişi ve mimari paradigması Overview başlığı altında özetlenir; ardından sistemin karşılaması gereken gereksinimler üç kategoride verilir: Functional Requirements (kabul kriterli), Nonfunctional Requirements (ölçülebilir) ve Pseudo Requirements (teknoloji/platform kısıtları).

2.1 Overview

AgentLens, çok ajanlı LLM sistemlerinde (MAS) ajanlar arasında gerçekleşen konuşma akışını gerçek zamana yakın biçimde izleyen, bu akışı standart bir veri formatına dönüştüren ve iletişim/koordinasyon kaynaklı başarısızlıkları otomatik olarak tespit edip açıklayan modüler bir analiz sistemidir. Sistem, mevcut araçların sağlayamadığı otomatik tespit ve açıklama katmanı eksikliğini kapatmayı hedefler: tespit edilen her uyuşmazlığın türünü (MAST taksonomisine göre), gerçekleştiği konuşma adımını, ilgili ajanları ve nedenini geliştiricinin manuel log incelemesine gerek kalmadan raporlar.

Sistem, izlenen çok ajanlı yapının iç işleyişine müdahale etmeyen (non-invasive), log kayıtları üzerinden çalışan bir dış araç olarak tasarlanmıştır. Önerilen mimari, her bileşenin tek bir sorumluluğa sahip olduğu modüler bir pipeline üzerine kuruludur. Veri akışı sırasıyla şu bileşenlerden geçer:

- **Data Collector** — Ajanlar arası mesajları, log kayıtlarını ve metadata'yı (gönderen/alıcı ajan, tur sırası, zaman damgası, araç çağırısı, görev bağlamı) yakalar.
- **Raw Trace Storage** — Ham konuşma kayıtlarını dönüştürmeden saklar; yeniden işleme ve geriye dönük analiz imkânı sağlar.
- **Data Formatter** — Farklı kaynaklardan gelen ham veriyi ortak, standart bir JSON konuşma şemasına dönüştürür.
- **Formatted Data Storage** — Formatlanmış veriyi saklar ve bir sonraki bileşene sunar.
- **Conversation Window Builder** — Güncel mesajı birikmiş konuşma geçmişiyle birleştirerek bağlama duyarlı bir analiz penceresi üretir.
- **Stage 1: Anomaly Detection Model** — Her pencereyi düşük maliyetli bir ön kontrolle "normal" veya "şüpheli" olarak sınıflandırır ve anomali skoru üretir.

- **Stage 2: Failure Analysis Model** — Yalnızca şüpheli pencereleri, MAST Store'dan retrieval destekli dinamik few-shot örneklerle daha güçlü bir modelle analiz ederek hata türünü, adını, ilgili ajanları ve nedenini belirler.
- **Report Generator** — Analiz sonucunu yapılandırılmış (structured) bir rapora dönüştürür.
- **Dashboard / Logs** — Normal akışları, şüpheli durumları ve detaylı hata raporlarını kullanıcıya sunar.

Sistemin ayırt edici tasarım kararı, tespit sürecinin iki aşamaya ayrılmasıdır. Tek aşamalı büyük bir LLM değerlendirmesi her konuşma adımında yüksek maliyet doğururken, yalnızca küçük bir sınıflandırıcı ise ayrıntılı ve açıklanabilir raporlama yapamazdı. İki aşamalı yapı; normal konuşmaları hızlı ve ucuz biçimde filtreler, yalnızca riskli pencereler için maliyetli ve ayrıntılı analizi devreye sokar. Böylece performans, maliyet ve açıklanabilirlik arasında denge kurulur. Modüler tasarım sayesinde herhangi bir bileşen değiştirildiğinde diğerleri etkilenmez ve sistem farklı çok ajanlı yapılarla kolayca entegre edilebilir; entegrasyonun mümkün olmadığı durumlarda ise sistem, dışarıdan JSON/JSONL konuşma kaydı alıp analiz eden bağımsız bir çevrimdışı araç olarak da çalışabilir.

2.2 Functional Requirements

Fonksiyonel gereksinimler işlevsel gruplara ayrılmıştır: (A) Veri Toplama ve Saklama, (B) Bağlam Oluşturma, (C) Tespit ve Analiz, (D) Raporlama ve Görselleştirme, (E) Veri Yönetimi ve Değerlendirme, (F) Gizlilik ve Güvenlik. Her gereksinim bir öncelik (Yüksek / Orta / Düşük) ve doğrulanabilir bir kabul kriteri (acceptance criteria) taşır.

2.2.1 Veri Toplama ve Saklama

FR-1 — Ajanlar Arası Veri Toplama Öncelik: Yüksek Data Collector, izlenen çok ajanlı LLM sisteminden ajanlar arası mesajları, log kayıtlarını ve ilgili metadata'yı (gönderen ajan, alıcı ajan, tur sırası, zaman damgası, araç çağrısı bilgisi, gerçekleştirilen aksiyon, görev bağlamı) yakalamalıdır.

Kabul Kriteri: Çalışan bir MAS test senaryosunda üretilen ajan mesajlarının en az %95'i yakalanır ve yakalanan her kayıt; gönderen ajan, alıcı ajan, tur sırası ve zaman damgası zorunlu alanlarını eksiksiz içerir. Bu dört alan bir kayıta eksikse kayıt "eksik veri" olarak işaretlenir.

FR-2 — Ham Veri Saklama Öncelik: Yüksek Sistem, toplanan ham veriyi hiçbir dönüşüm uygulamadan Raw Trace Storage içinde saklamalı ve gerektiğinde yeniden işlenebilmesine olanak tanımalıdır.

Kabul Kriteri: Kaydedilen bir oturum, session_id ile sorgulandığında orijinal mesaj sırasıyla ve içeriği değişmeden eksiksiz geri getirilebilir. Aynı oturum, formatlama hattı yeniden çalıştırıldığında ham kayıttan yeniden üretilebilir.

FR-3 — Veri Formatlama Öncelik: Yüksek Data Formatter, farklı kaynaklardan gelen ham trace verisini ortak ve standart bir JSON konuşma şemasına (ajan adları, roller, mesaj içeriği, konuşma sırası, global görev, araç çağrısı, aksiyon detayları) dönüştürmelidir.

Kabul Kriteri: En az iki farklı log biçiminden gelen ham veri, aynı JSON şemasına dönüştürülür ve şema doğrulaması (pydantic) hatasız geçer. Zorunlu alanı eksik olan kayıtlar dönüşümü durdurmaz; hata kayıt listesine düşürülür ve raporlanır.

FR-4 — Formatlı Veri Saklama Öncelik: Orta Sistem, formatlanmış konuşma verisini Formatted Data Storage içinde saklamalı ve Conversation Window Builder tarafından okunabilir kılmalıdır.

Kabul Kriteri: Formatlanmış bir oturum session_id ile Formatted Data Storage'dan okunabilir ve okunan veri, ilgili şema doğrulamasını geçer.

FR-5 — Yeniden İşleme (Reprocessing) Öncelik: Orta Sistem, formatlama veya pipeline aşamasında bir hata oluştuğunda ilgili oturumları Raw Trace Storage üzerinden yeniden işleyebilmelidir.

Kabul Kriteri: Bilinçli olarak bozulmuş bir formatlama adımı düzeltildikten sonra, ilgili oturum ham kayıttan yeniden işlendiğinde çıktı, hatasız senaryodaki çıktıyla birebir aynı olur.

2.2.2 Bağlam Oluşturma

FR-6 — Bağlam Penceresi Oluşturma Öncelik: Yüksek Conversation Window Builder, güncel mesajı birikmiş konuşma geçmişiyile birleştirerek bağlama duyarlı bir analiz penceresi üretmelidir.

Kabul Kriteri: Verilen bir güncel mesaj için, yapılandırılabilir pencere boyutu (son N mesaj) kadar geçmiş birleştirilerek tek bir analiz penceresi üretilir. Üretilen pencere mesajların kronolojik sırasını korur; pencere boyutu mevcut mesaj sayısından fazlaysa tüm mesajları içerir ve hata vermez.

2.2.3 Tespit ve Analiz

FR-7 — Anomali Tespiti (İkili Sınıflandırma) Öncelik: Yüksek Stage 1: Anomaly Detection modeli, her analiz penceresini few-shot prompting yöntemiyle "normal" veya "şüpheli" olarak ikili sınıflandırmalıdır.

Kabul Kriteri: Her analiz penceresi için Stage 1, normal veya şüpheli etiketlerinden tam olarak birini döndürür. Etiket boş, null veya tanımsız olamaz.

FR-8 — Anomali Skoru Öncelik: Orta Stage 1, kararına dair güven seviyesini gösteren bir anomali skoru üretmelidir.

Kabul Kriteri: Stage 1 her karar için $[0, 1]$ aralığında sayısal bir anomali skoru döndürür ve bu skor, yapılandırılabilir bir eşik değeriyle karşılaştırılabilir.

FR-9 — Normal Akış Yönlendirme Öncelik: Orta Normal olarak etiketlenen pencereler, Stage 2'yi devreye sokmadan doğrudan Dashboard/Logs bileşenine iletilmelidir.

Kabul Kriteri: normal etiketli bir pencere için Stage 2 modeline hiçbir API çağrısı yapılmaz ve pencere Dashboard/Logs kayıtlarında görünür.

FR-10 — Şüpheli Akış Yönlendirme Öncelik: Yüksek Şüpheli olarak etiketlenen pencereler, ileri analiz için Stage 2'ye iletilmelidir.

Kabul Kriteri: şüpheli etiketli her pencere Stage 2 girişine iletilir ve Stage 2 için bir analiz kaydı oluşturulur.

FR-11 — Dinamik Few-Shot Retrieval Öncelik: Yüksek Stage 2, MAST Store'dan embedding tabanlı retrieval ile o vakaya anlamsal olarak en yakın örnekleri dinamik olarak seçmeli ve prompt'una few-shot örnek olarak eklemelidir.

Kabul Kriteri: Her şüpheli pencere için MAST Store'dan embedding benzerliğine göre en yakın K (yapılandırılabilir) örnek getirilir ve Stage 2 prompt'una eklenir. Aynı girdi için retrieval sonucu tekrarlanabilir (deterministik) olmalıdır.

FR-12 — Hata Analizi Öncelik: Yüksek Stage 2: Failure Analysis modeli, her şüpheli pencere için hatanın türünü (ilgili MAST kategorisi/hata modu), hatanın başladığı konuşma adımını, hataya dahil olan ajanları ve hatanın nedenini belirlemelidir.

Kabul Kriteri: Stage 2 çıktısı, her şüpheli pencere için şu dört alanı da doldurur: (a) hata türü (MAST kategori/FM kodu), (b) hatanın başladığı konuşma adımı (tur numarası), (c) ilgili ajanlar listesi, (d) hatanın nedenine dair açıklama. Alanlardan hiçbirisi boş bırakılamaz.

2.2.4 Raporlama ve Görselleştirme

FR-13 — Rapor Üretimi Öncelik: Yüksek Report Generator, Stage 2 çıktısını hem insan hem de diğer sistemler tarafından işlenebilir, yapılandırılmış (structured output) bir rapor formatına dönüştürmelidir.

Kabul Kriteri: Üretilen her rapor, önceden tanımlı JSON şemasına uyar ve şema doğrulamasını geçer. Rapor hem makine tarafından ayrıştırılabilir alanları hem de insan tarafından okunabilir bir özet metnini içerir.

FR-14 — Görselleştirme ve Loglama Öncelik: Orta Dashboard/Logs bileşeni, normal etiketlenen konuşmaları, tespit edilen hataları ve üretilen raporları kullanıcıya görüntüleyebilmelidir.

Kabul Kriteri: Kullanıcı, Dashboard üzerinden analiz edilmiş oturumları listeleyebilir; bir hata kaydına tıkladığında hata türü, başladığı adım, ilgili ajanlar ve açıklama tek ekranda görüntülenir. Normal konuşmalar da aynı biçimde görüntülenebilir.

2.2.5 Veri Yönetimi ve Değerlendirme

FR-15 — Sentetik Veri Üretimi Öncelik: Orta Sistem, MAST veri kümesinin yetersiz kaldığı senaryolarda manuel veya yarı otomatik olarak oluşturulan etiketli sentetik konuşma örneklerinin sisteme dahil edilmesini desteklemelidir. **Kabul Kriteri:** Etiketli bir sentetik konuşma örneği, gerçek örneklerle aynı JSON şemasına uygun biçimde sisteme eklenebilir ve aynı analiz hattından (Stage 1 → Stage 2) hatasız geçer.

FR-16 — Performans Değerlendirme Öncelik: Yüksek Sistem, etiketli test kümesi üzerinde precision, recall, F1-skoru, accuracy ve confusion matrix metriklerini hesaplayabilmelidir.

Kabul Kriteri: Etiketli bir test kümesi verildiğinde sistem; precision, recall, F1-skoru, accuracy değerlerini ve confusion matrix'i üretir. Metrikler sayısal olarak raporlanır ve aynı girdi için tekrar çalıştırıldığında aynı sonucu verir.

2.2.6 Gizlilik ve Güvenlik

FR-17 — Gizlilik ve Takma Adlandırma Öncelik: Yüksek Sistem, harici LLM API'lerine veri göndermeden önce kişisel verileri takma adlandırmayı (token mapping / pseudonymization) ve üretilen raporlarda kişisel bilgilere yer vermemeyi desteklemelidir.

Kabul Kriteri: Belirlenen kişisel veri alanları (ör. kullanıcı adı, e-posta), harici API'ye gönderilen istekte anlamsal takma adlarla değiştirilir. Üretilen nihai raporlarda ham kişisel veri bulunmaz; bu, kişisel veri içeren bir test girdisiyle doğrulanır.

FR-18 — Veri Bütünlüğü Doğrulama Öncelik: Orta Sistem, zorunlu alanları (gönderen, alıcı, zaman damgası, tur sırası) doğrulamalı ve eksik veri durumunda kullanıcıyı uyarmalıdır.

Kabul Kriteri: Zorunlu alanlardan biri eksik olan bir kayıt işleme alındığında sistem bir uyarı üretir, kaydı "eksik veri" olarak işaretler ve pipeline'ı çökertmeden devam eder.

2.3 Nonfunctional Requirements

Fonksiyonel olmayan gereksinimler, ölçülebilir hedeflerle (sayı, oran, süre) tanımlanmış ve kalite kategorilerine göre gruplanmıştır.

2.3.1 Performans

NFR-1 — Oturum İşlem Süresi: Belirlenen maksimum konuşma penceresi boyutu için uçtan uca analiz ve raporlama süreci, oturum başına ortalama 10 saniyenin altında tamamlanmalıdır.

NFR-2 — Yerel İşleme Gecikmesi: Data Formatter ve Conversation Window Builder modüllerinin yerel işleme gecikmesi (harici LLM çağrılarını hariç) p95 < 500 ms olmalıdır.

2.3.2 Doğruluk ve Kalite

NFR-3 — Tespit Başarımı: Sistem, MAST veri kümesindeki iletişim ve koordinasyon uyumsuzluklarını tespit etmede en az 0,80 F1-skoru elde etmelidir.

NFR-4 — Sınıflandırma Doğruluğu: Tespit edilen uyumsuzlukların doğru MAST hata kategorisine atanmasında en az %75 sınıflandırma doğruluğu sağlanmalıdır.

NFR-5 — Açıklanabilirlik Tamlığı: Test edilen örneklerin en az %90'ında üretilen rapor; hatanın gerçekleştiği adımı, hata türünü ve nedene dair açıklamayı eksiksiz içermelidir.

NFR-6 — Yüksek Recall Önceliği: Stage 1, kaçırılan anomali (false negative) oranını en aza indirmek için yüksek recall önceliğiyle çalışmalıdır; false negative oranı, false positive oranından düşük tutulmalı ve recall değeri MAST referans değerlerine olabildiğince yaklaşmalıdır.

2.3.3 Modülerlik ve Bakım

NFR-7 — Modülerlik: Her modül tek bir sorumluluğa sahip olmalı, tanımlı input/output arayüzleri üzerinden bağımsız olarak değiştirilebilmelidir. Bir modülün yeniden yazılması, komşu modüllerin kaynak kodunda değişiklik gerektirmemelidir.

2.3.4 Taşınabilirlik ve Entegrasyon

NFR-8 — Dışarıdan Bağlanabilirlik: Sistem, izlenen çok ajanlı sistemi değiştirmeden log kayıtları üzerinden çalışmalı; ayrıca JSON/JSONL formatındaki konuşma kayıtlarını kabul ederek çevrimdışı (offline) analiz yapabilmelidir.

NFR-9 — Genişletilebilir Şema: Veri şeması minimum zorunlu alanlardan oluşmalı, ek alanlar opsiyonel olmalıdır; böylece farklı framework'lerden gelen ve yalnızca temel alanları içeren veriler de analiz edilebilmelidir.

2.3.5 Güvenlik ve Gizlilik

NFR-10 — Erişim ve Veri Koruma: Raw Trace Storage ve Formatted Data Storage içindeki verilere yalnızca yetkili proje üyeleri erişebilmeli; açık kaynak depo paylaşılmadan önce veriler gerçek kullanıcı verisi, API anahtarı veya kurum içi gizli bilgi içermemelidir.

NFR-11 — Anahtar Yönetimi: API anahtarları environment variable veya secret management mekanizması üzerinden yönetilmeli, .env dosyaları .gitignore kapsamına alınmalıdır. Depoda hiçbir düz metin API anahtarı bulunmamalıdır.

2.3.6 Güvenilirlik

NFR-12 — Yeniden İşleme Dayanıklılığı: Formatlama veya pipeline aşamasında bir hata oluştuğunda sistem, ilgili konuşmaları Raw Trace Storage üzerinden veri kaybı olmadan yeniden işleyebilmelidir.

NFR-13 — Veri Bütünlüğü: Sistem, zorunlu alanları (gönderen, alıcı, zaman damgası, tur sırası) doğrulamalı ve eksik veri durumunda pipeline'ı durdurmadan uyarı üretmelidir.

2.3.7 Verimlilik ve Çevresel Etki

NFR-14 — Maliyet/Enerji Optimizasyonu: Yüksek maliyetli Stage 2 modeli yalnızca anomali etiketli pencerelerde çalıştırılmalıdır; toplam Stage 2 çağrısı sayısı, toplam pencere sayısından belirgin biçimde (hedef: %50'den fazla) düşük olmalıdır.

2.3.8 Kullanılabilirlik

NFR-15 — İşlenebilir Çıktı: Üretilen raporlar structured output (JSON) formatında olmalı; hem insanlar hem de dış sistemler tarafından işlenebilmelidir.

2.4 Pseudo requirements

Pseudo gereksinimler, sistemin *ne yaptığını* değil, hangi teknoloji, platform ve süreç kısıtları altında geliştirileceğini tanımlar. Bu kısıtlar takımın teknik kararlarından ve projenin akademik/ekonomik bağlamından kaynaklanır.

PR-1 — Geliştirme Dili: Sistem birincil olarak Python 3.x ile geliştirilecektir; tüm modüller Python ekosistemine dayanır.

PR-2 — Harici LLM Sağlayıcıları: Stage 1 ve Stage 2 modelleri, harici ticari LLM API'leri üzerinden çalışır. Birincil olarak düşük maliyetli/hızlı modeller (GPT-4o-mini veya Claude Haiku) kullanılır; gerektiğinde daha güçlü modeller (GPT-4o / Claude 3.5 Sonnet) yalnızca Stage 2 için opsiyonel olarak devreye alınabilir.

PR-3 — Şema Doğrulama: Veri şeması doğrulaması ve structured data validation için pydantic; garantili JSON çıktısı için instructor kullanılabilir.

PR-4 — Retrieval Altyapısı: Embedding tabanlı dinamik few-shot seçimi için sentence-transformers ve vektör benzerlik araması için FAISS veya ChromaDB kullanılabilir.

PR-5 — Arayüz Teknolojisi: Dashboard/Logs bileşeni Streamlit veya Dash ile geliştirilebilir; ağır bir front-end framework'e bağımlılık hedeflenmemektedir.

PR-6 — Veri Değişim Formatı: Modüller arası ve çevrimdışı analiz için birincil veri değişim formatı JSON/JSONL'dir.

PR-7 — Müdahalesiz İzleme: Sistem, izlenen çok ajanlı MAS'ın iç işleyişine müdahale etmeden, yalnızca log kayıtları ve dış gözlem üzerinden çalışmalıdır (non-invasive monitoring).

PR-8 — Yapılandırılabilirlik: Sınıflandırma eşikleri, pencere boyutu, retrieval örnek sayısı (K) ve referans veri seti gibi parametreler koda gömülmek (hardcoded) yerine dışarıdan yapılandırılabilir (configurable) olmalıdır.

PR-9 — Versiyon Kontrolü ve Geliştirme Ortamı: Ekip içi kod paylaşımı için bir GitHub organizasyonu, model geliştirme aşamasında ise ağırlıklı olarak Google Colab kullanılır.

PR-10 — Lisans Uyumluluğu: Kullanılan tüm üçüncü taraf kütüphaneler açık kaynak lisanslarına (MIT, Apache 2.0, BSD, PSF vb.) uygun olmalı ve açık kaynak depo paylaşımından önce THIRD_PARTY_NOTICES altında belgelenmelidir.

PR-11 — Bütçe Kısıtı: Proje kurumsal bütçe veya araştırma fonu desteği olmadan yürütüldüğünden, geliştirme ve test sürecinde sağlayıcıların ücretsiz/öğrenci kotaları esas alınır; deneme sayısı ve model kapasitesi bu bütçeyle sınırlıdır.

3. System models

3.1 Scenarios

Bu doküman, AgentLens projesi kapsamında sistemin çalışma zamanı davranışlarını ve olası kullanıcı etkileşimlerini tanımlayan senaryoları içermektedir. Belge, normal işleyişi gösteren "Mutlu Yollar" ve istisnai durumları ele alan "Hata Yolları" olmak üzere iki ana bölüme ayrılmıştır.

3.1.1 Mutlu Yol Senaryoları (Happy Paths)

Senaryo 1: Gerçek Zamanlı Sorunsuz Konuşma Akışı

Senaryo: Çok ajanlı sistemde (MAS) ajanlar arasında iletişim kopukluğu olmadan, normal bir şekilde gerçekleşen diyalogların sistem tarafından işlenip filtrelenmesi akışıdır.

Aktörler: Çok Ajanlı Sistem (MAS), AgentLens Sistemi, Kullanıcı (Yazılım Geliştirici / Araştırmacı).

Adım	Bileşen	Aksiyon / Beklenen Sonuç
1	Data Collector	Ajanlar arası mesaj döngüsünü, log kayıtlarını ve metadata bilgilerini anlık olarak yakalar ve Raw Trace Storage'a kaydeder.
2	Data Formatter	Ham kayıtları okuyarak standart JSON konuşma şemasına dönüştürür.
3	Conversation Window Builder	Güncel mesajı konuşma geçmişiyle birleştirip analiz

Adım	Bileşen	Aksiyon / Beklenen Sonuç
		penceresi üretir.
4	Stage 1: Anomaly Detection	Pencereyi analiz eder ve "normal" olarak sınıflandırır.
5	Dashboard / Logs	Analiz, Stage 2'ye gitmeden doğrudan "Normal" etiketiyle arayüzde listelenir.

Senaryo 2: Gerçek Zamanlı Hata Tespiti ve Raporlanması

Senaryo: Ajanlar arası iletişim ve koordinasyon kopukluğu yaşanması durumunda tespit ve raporlama akışını tanımlar.

Aktörler: Çok Ajanlı Sistem (MAS), AgentLens Sistemi, Kullanıcı (Yazılım Geliştirici / Araştırmacı).

Adım	Bileşen	Aksiyon / Beklenen Sonuç
1	MAS & Pipeline	Uyumsuzluk içeren akış toplanır, formatlanır ve analiz penceresi oluşturulur.
2	Stage 1: Anomaly Detection	Pencereyi "şüpheli" olarak etiketler ve ileri analize iletir.
3	Stage 2: Failure Analysis	MAST Store üzerinden en yakın geçmiş örnekleri (retrieval) çeker ve prompt'una ekler.
4	Stage 2: Failure Analysis	Hata türünü, ilgili adımı, ajanları ve hatanın nedenini belirler.
5	Report Generator & Dashboard	Analizi yapılandırılmış rapora dönüştürür ve kullanıcı arayüzünde detaylı olarak sunar.

Senaryo 3: Çevrimdışı (Offline) Log Analizi

Senaryo: Farklı bir MAS üzerinden alınmış eski konuşma loglarının çevrimdışı işlenmesi adımlarını gösterir.

Aktörler: Kullanıcı (Yazılım Geliştirici / QA Uzmanı), AgentLens Sistemi.

Adım	Bileşen	Aksiyon / Beklenen Sonuç
1	Kullanıcı & Data Formatter	Dışarıdan yüklenen JSON/JSONL geçmiş kayıtları sisteme beslenir ve standart şemaya dönüştürülür.
2	Conversation Window Builder	Formatted Data Storage'dan okunan veriler analiz pencerelerine ayrılır.
3	Stage 1 & Stage 2	Konuşmalar normal analiz veya anomali tespiti (gerekirse Stage 2) üzerinden geçirilir.
4	Dashboard / Logs	Sonuçlar, geçmiş oturumun hata dökümü olarak detaylıca listelenir.

Senaryo 4: Sentetik Veri ile Performans Değerlendirmesi (Yeni)

Senaryo: Sistemin performansının etiketli sentetik veri kümeleri üzerinden test edilmesi ve raporlanması akışıdır.

Aktörler: Kullanıcı (QA Uzmanı / Araştırmacı), AgentLens Sistemi.

Adım	Bileşen	Aksiyon / Beklenen Sonuç
1	Kullanıcı	MAST kümesinin yetersiz kaldığı durumlar için oluşturulan etiketli sentetik verileri sisteme yükler.
2	Pipeline İşleme	Sentetik örnekler JSON şemasına formatlanır ve bağlam pencereleri oluşturulur.
3	Değerlendirme Modülü	Pencereler Stage 1 ve Stage 2'den geçirilir, model sonuçları test etiketleri ile karşılaştırılır.
4	Raporlama	Sistem; precision, recall, F1-skoru, accuracy ve confusion matrix metriklerini hesaplayıp arayüzde listeler.

3.1.2 Hata Yolu Senaryoları (Error Paths)

Hata Yolu 1: Zorunlu Alanı Eksik Log Kaydının İşlenmesi

Senaryo: Zorunlu veri alanlarının log kayıtlarında bulunmaması durumunda sistemin veri bütünlüğünü koruma adımlarını tanımlar.

Aktörler: AgentLens Sistemi, Çok Ajanlı Sistem (MAS).

Adım	Bileşen	Aksiyon / Beklenen Sonuç
1	Data Collector	Zorunlu alanlardan (gönderen, alıcı, zaman damgası vb.) birinin eksik olduğu mesajı Raw Trace Storage'a kaydeder.
2	Data Formatter	Veriyi formatlarken zorunlu alan eksikliğini fark eder ve kaydı "eksik veri" olarak işaretler.
3	Pipeline Kontrolü	Sistem çökmek yerine çalışmaya devam eder ve eksik veri durumunda uyarı kaydı oluşturur.
4	Dashboard / Logs	Kullanıcıya veri bütünlüğü eksikliği uyarısı raporlanır.

Hata Yolu 2: Formatlama Hatası ve Yeniden İşleme (Yeni)

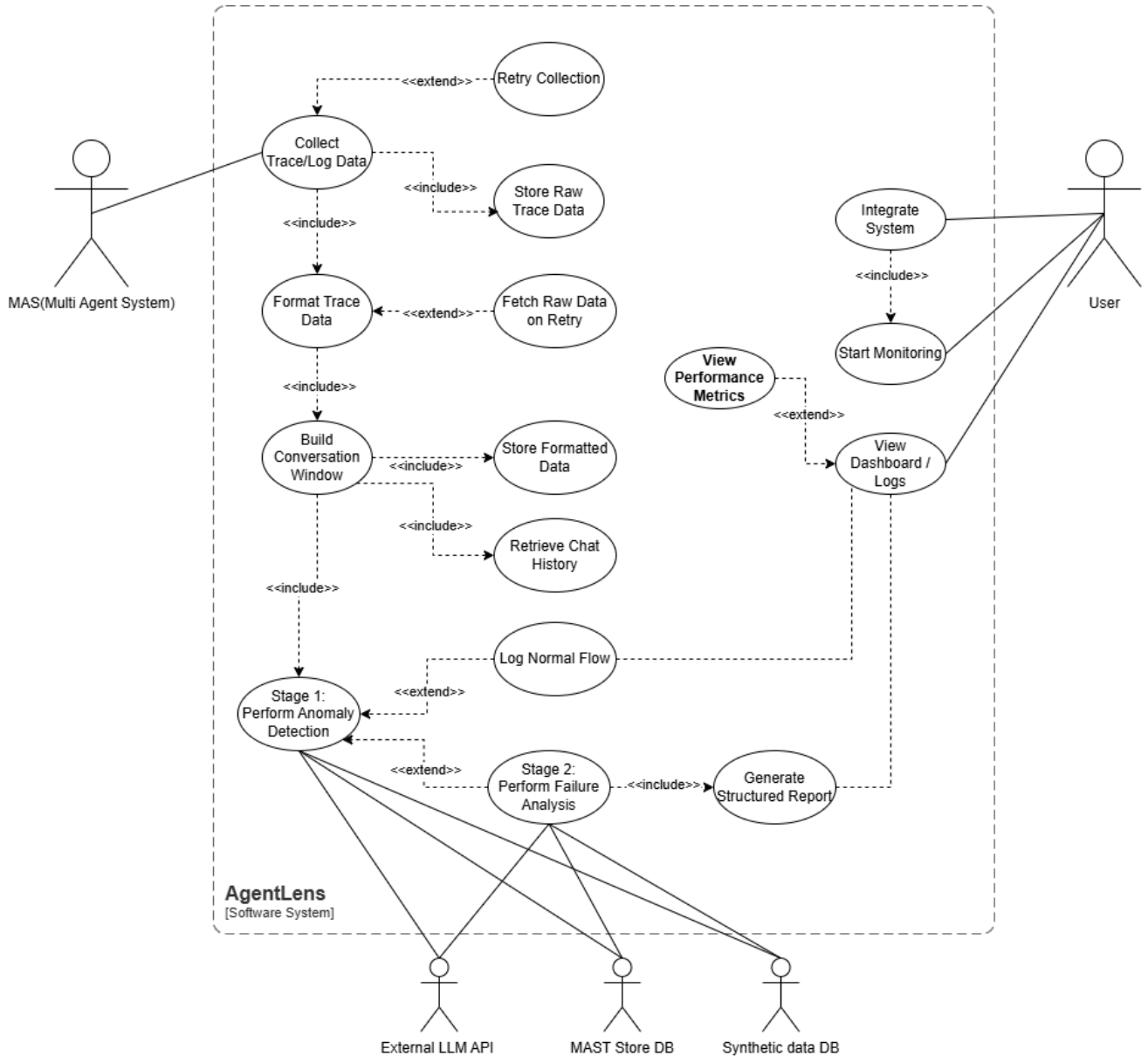
Senaryo: Analiz hattında oluşan beklenmedik teknik veya formatlama hatalarının, ham veri korunarak geri döndürülmesi ve tekrar işlenmesi akışıdır.

Aktörler: AgentLens Sistemi, Sistem Yöneticisi / Geliştirici.

Adım	Bileşen	Aksiyon / Beklenen Sonuç
1	Pipeline Hata Durumu	Data Formatter veya Window Builder aşamasında beklenmedik bir hata (exception) oluşur ve akış kesilir.
2	Raw Trace Storage	Hata loglanır, ancak oturumun orijinal (ham) hali Raw Trace Storage içerisinde veri kaybı olmadan korunur.
3	Sistem Müdahalesi	Hata teknik olarak düzeltildikten sonra, ilgili oturum (session_id)

Adı m	Bileşen	Aksiyon / Beklenen Sonuç
		sistemden "yeniden işle" komutu ile tekrar çağrılır.
4	Yeniden İşleme & Sonuç	Konuşma verisi Raw Trace Storage'dan alınarak doğru formatta saklanır ve analizi Dashboard'a sorunsuzca yansıtılır.

3.2 Use Case Model



UC-01 Collect Trace/Log Data: Sistemin, dış ortamda çalışan ajanların ürettiği ham mesajları, aksiyonları ve log verilerini dinleyip yakalaması sürecidir.

- **Aktörler:** MAS (Multi Agent System)
- **İlişkiler:**
 - <<include>> Store Raw Trace Data: Yakalanan veri veritabanına yazılır.
 - <<extend>> Retry Collection: Veri çekilirken bağlantı kopması veya zaman aşımı olursa sistem veriyi yeniden çekmeyi dener.
 - <<include>> Format Trace Data: Toplanan veri yapılandırılmak üzere bir sonraki aşamaya iletilir.

UC-02 Retry Collection: Veri çekilirken bağlantı kopması veya zaman aşımı olması durumunda sistemin veri toplamayı otomatik olarak yeniden denemesi işlemidir.

- **Aktörler:** Yok (Sistem içi eylem)

UC-03 Store Raw Trace Data: Yakalanan ham verinin kaybolmaması için anında kalıcı veritabanına yazılmasıdır.

- **Aktörler:** Yok (Sistem içi eylem)

UC-04 Format Trace Data: Toplanan ham log verilerinin AgentLens sisteminin işleyebileceği standart bir yapılandırılmış formata dönüştürülmesi işlemidir.

- **Aktörler:** Yok (Sistem içi eylem)
- **İlişkiler:**
 - <<include>> Build Conversation Window: Formatlanan veri, bağlam oluşturmak üzere bir sonraki adıma iletilir.
 - <<extend>> Fetch Raw Data on Retry: Formatlama sırasında hata oluşursa, sistem ham veriyi veritabanından tekrar çeker.

UC-05 Fetch Raw Data on Retry: Formatlama aşamasında veri eksikliği veya bozulma tespit edilirse, sistemin ham veriyi veritabanından yeniden çağırmasıdır.

- **Aktörler:** Yok (Sistem içi eylem)

UC-06 Build Conversation Window: Formatlanan güncel mesajın, ajanın geçmiş konuşma loglarıyla birleştirilerek yapay zeka analizine uygun bir bağlam penceresi haline getirilmesidir.

- **Aktörler:** Yok (Sistem içi eylem)
- **İlişkiler:**
 - <<include>> Store Formatted Data: Oluşturulan formatlı veri güvenli şekilde kaydedilir.
 - <<include>> Retrieve Chat History: Bağlam penceresini oluşturmak için geçmiş konuşmalar veritabanından çekilir.
 - <<include>> Stage 1: Perform Anomaly Detection: Oluşturulan bağlam penceresi analiz edilmek üzere yapay zeka modeline iletilir.

UC-07 Store Formatted Data: Yapay zeka modellerinin işleyebilmesi için oluşturulan bağlam penceresinin ve yapılandırılmış verinin sisteme kaydedilmesidir.

- **Aktörler:** Yok (Sistem içi eylem)

UC-08 Retrieve Chat History: Bağlam penceresi oluşturulurken ajanın geçmiş konuşma ve etkileşim verilerinin veritabanından getirilmesidir.

- **Aktörler:** Yok (Sistem içi eylem)

UC-09 Stage 1: Perform Anomaly Detection: Oluşturulan bağlam penceresinin LLM servisi kullanılarak analiz edilmesi ve iletişimde bir anomali olup olmadığının tespit edilmesidir. Karar mekanizmasının ilk adımıdır.

- **Aktörler:** External LLM API, MAST Store DB, Synthetic Data DB
- **İlişkiler:**
 - <<extend>> Log Normal Flow: Analizde anomali bulunmazsa, verinin normal olarak etiketlenip loglanması sağlar.
 - <<extend>> Stage 2: Perform Failure Analysis: Anomali tespit edilirse detaylı inceleme aşamasına geçişi tetikler.

UC-10 Log Normal Flow: UC-09 analizinde herhangi bir anomali bulunmazsa, verinin normal olarak etiketlenip sistem loglarına ve arayüze aktarılmasıdır.

- **Aktörler:** Yok (Çıktı Arayüze İletilir)

UC-11 Stage 2: Perform Failure Analysis: Anomali tespit edilmesi durumunda devreye giren detaylı inceleme aşamasıdır. Sistem; veritabanlarındaki destekleyici örnekleri çekerek hatanın türünü ve nedenini sınıflandırır.

- **Aktörler:** External LLM API, MAST Store DB, Synthetic Data DB
- **İlişkiler:**
 - <<include>> Generate Structured Report: Hata analizi gerçekleştirildiğinde yapılandırılmış rapor oluşturulması zorunludur.

UC-12 Generate Structured Report: UC-11'den elde edilen detaylı hata analizi sonuçlarının, kullanıcıların okuyabileceği yapılandırılmış bir rapor formatına dönüştürülmesidir.

- **Aktörler:** Yok (Çıktı Arayüze İletilir)

UC-13 Integrate System: Kullanıcının, AgentLens sistemini kendi MAS altyapısına bağlaması ve ortam yapılandırmalarını gerçekleştirmesidir.

- **Aktörler:** User
- **İlişkiler:**
 - <<include>> Start Monitoring: Sistem entegre edildikten sonra izleme işlemi zorunlu olarak başlar.

UC-14 Start Monitoring: Kullanıcının, MAS üzerinden akan verilerin dinlenmesini ve izleme hattını aktif hale getirmesidir.

- **Aktörler:** User

UC-15 View Dashboard / Logs: Kullanıcının sisteme giriş yaparak normal log akışlarını ve yapılandırılmış hata raporlarını arayüz üzerinden görüntülemesidir.

- **Aktörler:** User
- **İlişkiler:**
 - <<extend>> View Performance Metrics: Kullanıcı log ekranındayken detaylı metrik paneline opsiyonel olarak geçiş yapabilir.

UC-16 View Performance Metrics: Kullanıcının dashboard üzerinden ajanların genel performansını ve istatistiksel metrikleri incelemesidir.

- **Aktörler:** User

3.3 Object and Class Model

Bu bölümde AgentLens sistemine ait UML class diagram sunulmaktadır.

Class diagram, [Draw.io](https://draw.io) aracı kullanılarak oluşturulmuştur. Diyagramın üretiminde Mermaid formatında hazırlanan kaynak kod kullanılmış; bu kod [Draw.io](https://draw.io) üzerinde Insert → Mermaid seçeneği ile içe aktarılmıştır. Böylece sınıflar, attribute'lar ve ilişkiler UML class diagram formatında görselleştirilmiştir. Diyagramın sayfa düzeni nedeniyle okunabilirliğinin azalması durumunda, tam çözünürlüklü sürümüne aşağıdaki bağlantı üzerinden erişilebilir: https://drive.google.com/file/d/1QaPpe9ZoVkFmkCW0E4OFJN2lgFTAtcSG/view?usp=share_link

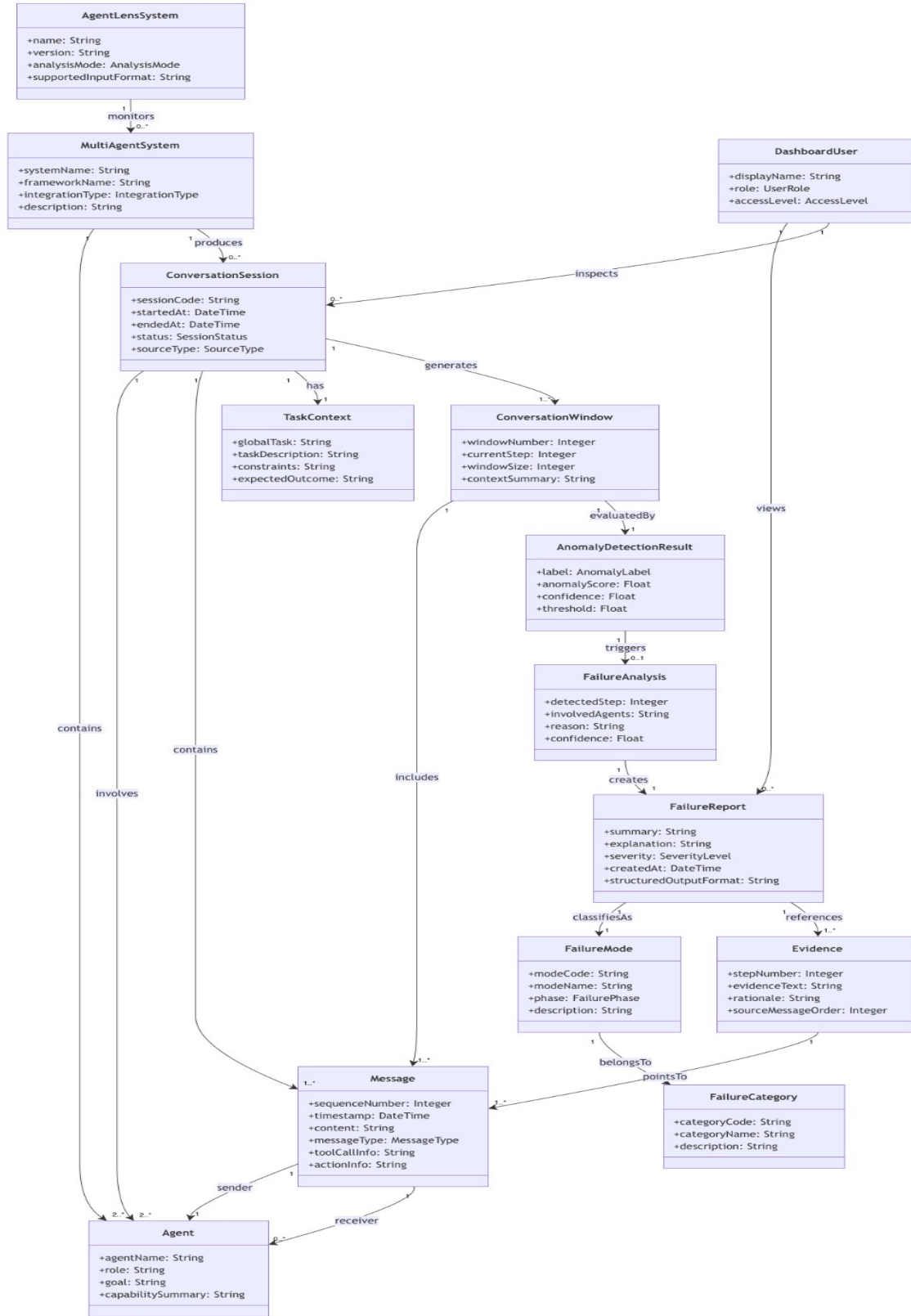
Kullanılan kaynak kodun bir bölümü aşağıda örnek olarak verilmiştir:

```
class Evidence {
+stepNumber: Integer
+evidenceText: String
+rationale: String
+sourceMessageOrder: Integer
}

class DashboardUser {
+displayName: String
+role: UserRole
+accessLevel: AccessLevel
}

AgentLensSystem "1" --> "0..*" MultiAgentSystem : monitors
MultiAgentSystem "1" --> "2..*" Agent : contains
MultiAgentSystem "1" --> "0..*" ConversationSession : produces

ConversationSession "1" --> "2..*" Agent : involves
ConversationSession "1" --> "1..*" Message : contains
ConversationSession "1" --> "1" TaskContext : has
```

Diyagramda Kullanılan Sınıflar ve Attribute'ların Açıklamaları:

AgentLensSystem sınıfı, geliştirilen analiz sistemini temsil etmektedir. name attribute'u sistemin adını, version attribute'u kullanılan sistem sürümünü, analysisMode attribute'u sistemin gerçek zamanlı, yakın gerçek zamanlı veya çevrimdışı analiz modunda çalıştığını, supportedInputFormat attribute'u ise sistemin desteklediği konuşma/log veri formatını ifade etmektedir.

MultiAgentSystem sınıfı, AgentLens tarafından izlenen çok ajanlı yapay zekâ sistemini temsil etmektedir. systemName attribute'u izlenen sistemin adını, frameworkName attribute'u sistemin hangi çok ajanlı framework veya yapı üzerinde geliştirildiğini, integrationType attribute'u AgentLens'in bu sisteme nasıl entegre edildiğini, description attribute'u ise izlenen sistemin genel amacını açıklamaktadır.

Agent sınıfı, çok ajanlı sistem içinde görev alan her bir ajanı temsil etmektedir. agentName attribute'u ajanın adını, role attribute'u ajanın sistem içindeki görev veya rolünü, goal attribute'u ajanın ulaşmaya çalıştığı amacı, capabilitySummary attribute'u ise ajanın temel yeteneklerini özetlemektedir. Bu bilgiler, ajanlar arası rol uyumsuzluğu, görev sapması veya koordinasyon hatalarının analiz edilebilmesi için gereklidir.

ConversationSession sınıfı, ajanlar arasında gerçekleşen bir konuşma oturumunu temsil etmektedir. sessionCode attribute'u oturumu ayırt etmek için kullanılan domain seviyesindeki tanımlayıcıdır. startedAt ve endedAt attribute'ları oturumun başlangıç ve bitiş zamanlarını gösterir. status attribute'u oturumun devam ediyor, tamamlandı veya analiz edildi gibi durumunu belirtir. sourceType attribute'u ise konuşmanın canlı sistemden, veri setinden veya sentetik test ortamından gelip gelmediğini ifade eder.

Message sınıfı, ajanlar arasında gerçekleşen tekil mesajları temsil etmektedir. sequenceNumber attribute'u mesajın konuşma sırasındaki konumunu gösterir ve hatanın hangi adımda oluştuğunu belirlemek için kullanılır. timestamp mesajın zaman bilgisini, content mesajın metinsel içeriğini, messageType mesajın normal mesaj, sistem mesajı, araç çağrısı veya aksiyon çıktısı gibi türünü ifade eder. toolCallInfo mesajla ilişkili araç çağrısı bilgisini, actionInfo ise ajanın gerçekleştirdiği eyleme ait bilgileri belirtir.

TaskContext sınıfı, konuşmanın bağlı olduğu görev bağlamını temsil etmektedir. globalTask attribute'u oturumun genel görevini, taskDescription attribute'u görevin açıklamasını, constraints attribute'u görev sırasında uyulması gereken kısıtları, expectedOutcome attribute'u ise görevden beklenen sonucu ifade etmektedir. Bu bilgiler, görev sapması veya hatalı doğrulama gibi durumların analiz edilmesinde kullanılır.

ConversationWindow sınıfı, analiz için oluşturulan konuşma penceresini temsil etmektedir. windowNumber attribute'u oturum içindeki pencere sırasını, currentStep attribute'u analiz edilen güncel konuşma adımını, windowSize attribute'u pencereye dahil edilen mesaj sayısını, contextSummary attribute'u ise önceki konuşma bağlamının kısa özetini ifade eder. Bu sınıf, tek bir mesajın değil, mesajın önceki konuşma geçmişiyle birlikte değerlendirilmesini sağlar.

AnomalyDetectionResult sınıfı, bir konuşma penceresi üzerinde yapılan ilk aşama anomali tespit sonucunu temsil etmektedir. label attribute'u pencerenin normal veya şüpheli olarak sınıflandırıldığını gösterir. anomalyScore attribute'u konuşma penceresinin anomali derecesini, confidence attribute'u modelin kararına duyduğu güveni, threshold attribute'u ise normal/şüpheli ayrımının hangi eşik değere göre yapıldığını ifade eder.

FailureAnalysis sınıfı, şüpheli olarak işaretlenen konuşma pencereleri üzerinde yapılan detaylı hata analizini temsil etmektedir. detectedStep attribute'u hatanın başladığı konuşma adımını, involvedAgents attribute'u

hataya dahil olan ajanları, reason attribute’u hatanın neden oluştuğuna dair açıklamayı, confidence attribute’u ise analiz sonucunun güven düzeyini ifade eder.

FailureReport sınıfı, hata analizi sonucunda kullanıcıya sunulan raporu temsil etmektedir. summary attribute’u raporun kısa özetini, explanation attribute’u hataya ilişkin detaylı açıklamayı, severity attribute’u hatanın önem düzeyini, createdAt attribute’u raporun oluşturulma zamanını, structuredOutputFormat attribute’u ise raporun yapılandırılmış çıktı formatını ifade etmektedir.

FailureCategory sınıfı, MAST taksonomisindeki ana hata kategorilerini temsil etmektedir. categoryCode attribute’u kategorinin kodunu, categoryName attribute’u kategorinin adını, description attribute’u ise kategorinin açıklamasını ifade eder. Bu sınıf, hata modlarının daha üst seviyedeki hata gruplarıyla ilişkilendirilmesini sağlar.

FailureMode sınıfı, MAST taksonomisinde tanımlanan belirli hata türlerini temsil etmektedir. modeCode attribute’u hata modunun kodunu, modeName attribute’u hata modunun adını, phase attribute’u hatanın pre-execution, execution veya post-execution aşamalarından hangisinde ortaya çıktığını, description attribute’u ise hata modunun açıklamasını ifade eder.

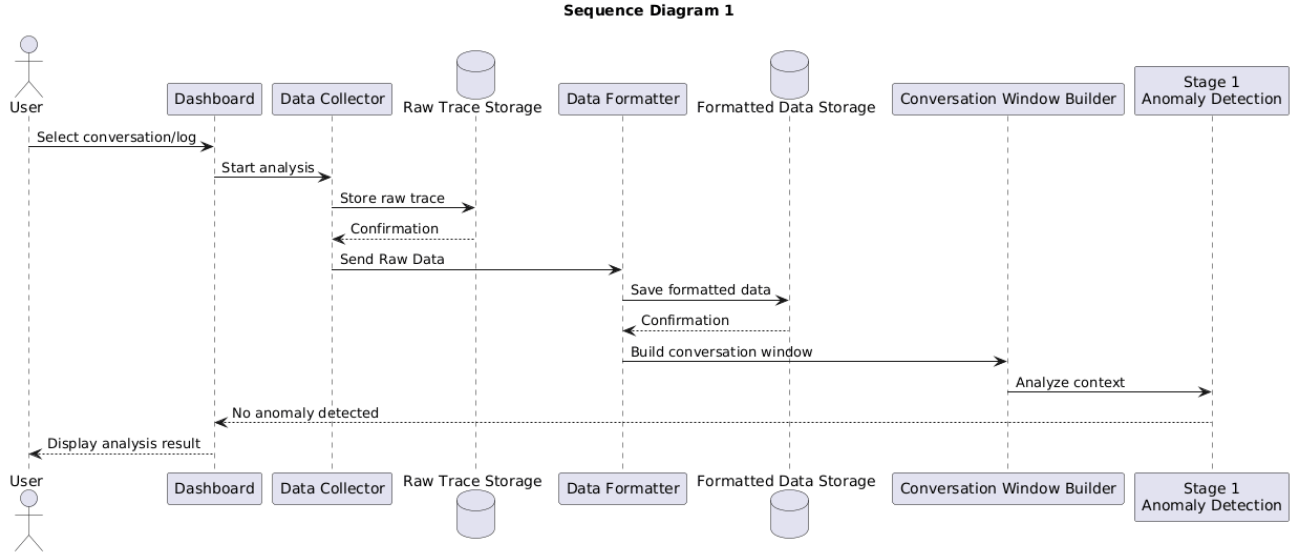
Evidence sınıfı, hata raporunda kullanılan kanıtları temsil etmektedir. stepNumber attribute’u kanıtın hangi konuşma adımına ait olduğunu, evidenceText attribute’u kanıt olarak kullanılan mesaj parçasını, rationale attribute’u bu mesajın neden kanıt olarak değerlendirildiğini, sourceMessageOrder attribute’u ise kanıtın konuşma sırasındaki yerini ifade eder. Bu sınıf, raporların açıklanabilir ve izlenebilir olmasını sağlar.

DashboardUser sınıfı, AgentLens arayüzünü kullanan kişiyi temsil etmektedir. displayName attribute’u kullanıcının arayüzde görünen adını, role attribute’u kullanıcının araştırmacı veya geliştirici gibi rolünü, accessLevel attribute’u ise kullanıcının hangi oturumlara veya raporlara erişebileceğini belirtir.

3.4 Dynamic Models

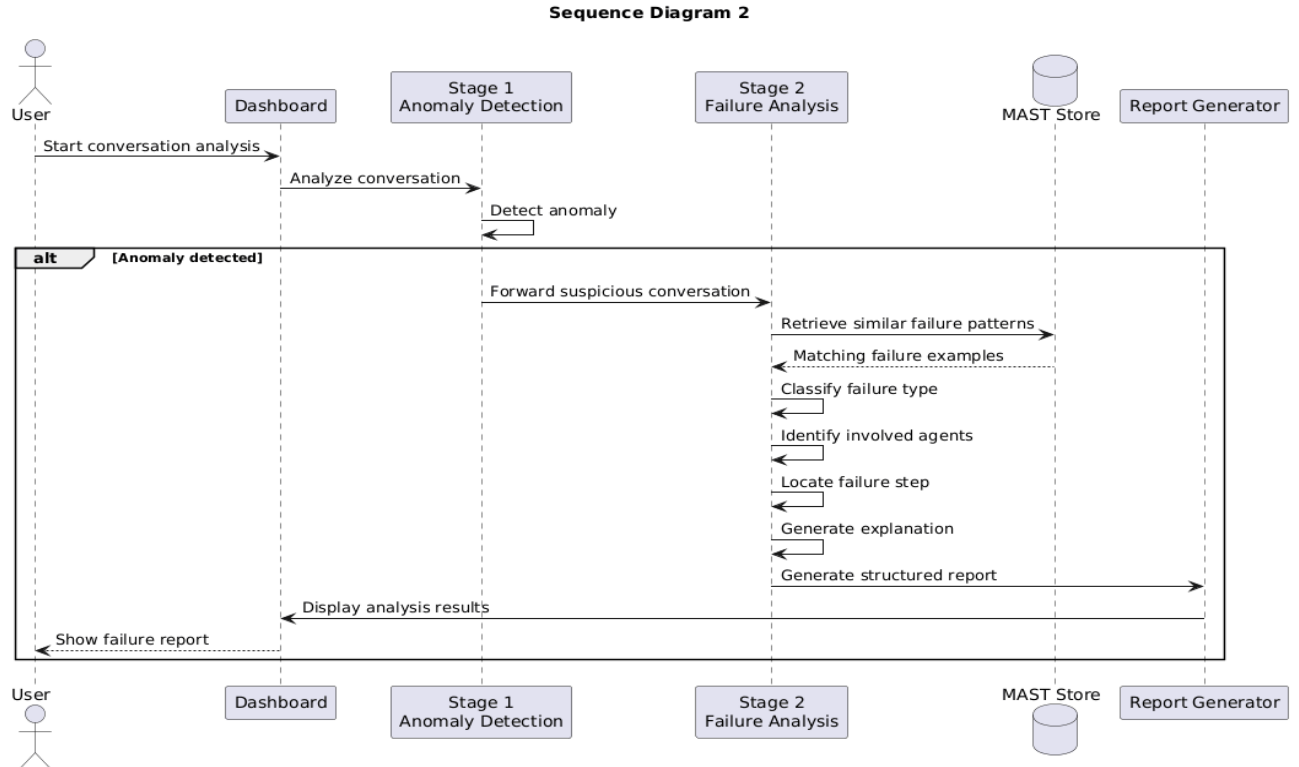
Bu bölümde sistemin runtime davranışlarını ve sistem bileşenleri arasındaki etkileşimleri gösteren dinamik modeller bulunmaktadır. Bu kapsamda geliştirilen sistem için iki adet sequence diyagramı ve bir adet activity diyagramı hazırlanmıştır. Sequence diyagramları sistem bileşenleri arasındaki etkileşimi gösterirken aktivite diyagramı sistemin genel akışını göstermektedir. Tüm dinamik modeller PlantUML kullanılarak hazırlanmıştır.

Sequence Diagram 1 - Konuşma Analizi:



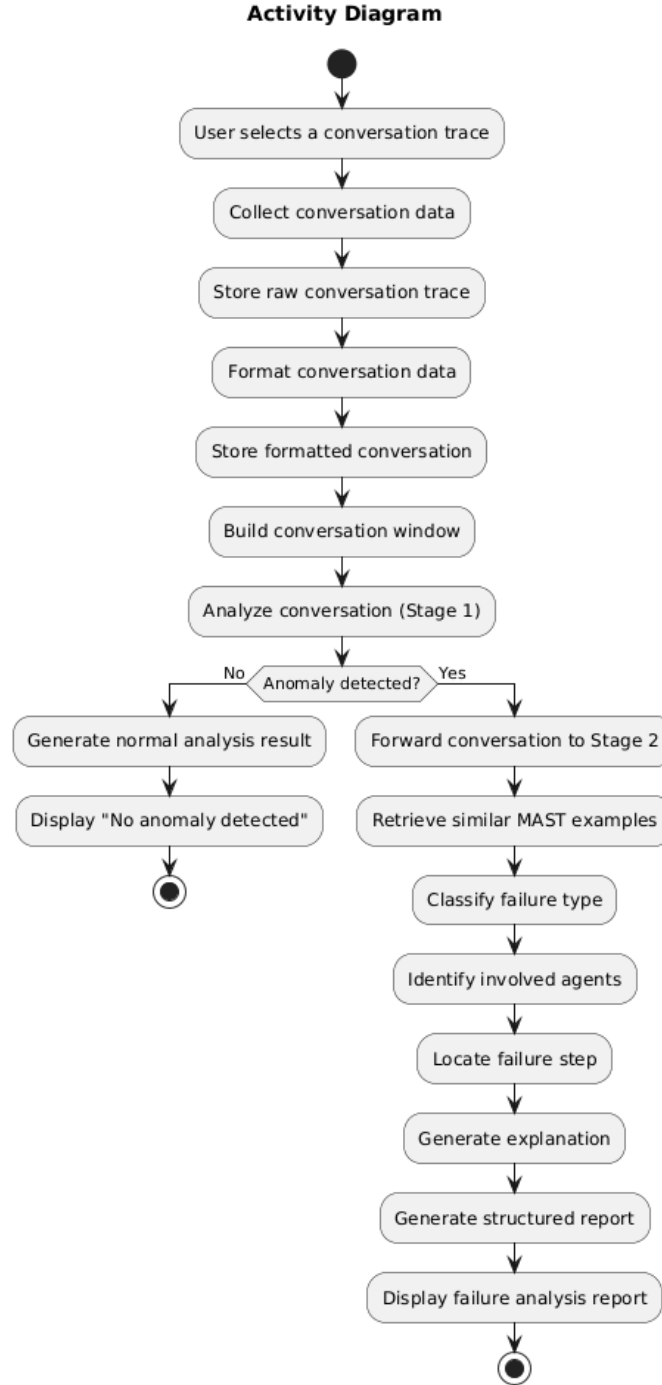
Birinci sequence diyagramı, sisteme gönderilen bir konuşma kaydının normal analiz sürecini göstermektedir. Süreç kullanıcının analiz edilmesi için bir konuşma kaydı seçmesiyle başlamaktadır. Ardından sistem konuşma verisini toplar, ham veriyi saklar, standart formata dönüştürür ve konuşma bağlamı penceresini oluşturur. Oluşturulan pencere analiz modülü tarafından incelenir. Bu aşamada herhangi bir problem tespit edilemediğinde analiz sonlandırılır ve sonuç kullanıcıya gösterilir.

Sequence Diagram 2 - Hata Analizi ve Raporlama:



İkinci sequence diyagramı, ilk analiz aşamasında bir problem tespit edildiğinde sistemin gerçekleştirdiği işlemleri göstermektedir. Stage 1 modülü tarafından şüpheli bulunan konuşma kaydı Stage 2 modülüne aktarılır. Sistem MAST veritabanından benzer örnekleri kullanarak hata türünü belirler, hataya neden olan ajanları ve hatanın gerçekleştiği adımı tespit eder. Elde edilen bilgiler kullanılarak okunabilir bir rapor oluşturup kullanıcıya sunulur.

Activity Diagram - Genel Sistem Akışı:



Activity diyagramı sistemin başlangıçtan sonuca kadar izlediği genel iş akışını göstermektedir. Süreç, konuşma kaydının seçilmesiyle başlamakta; veri toplama, veri biçimlendirme ve konuşma penceresinin oluşturulması adımlarıyla devam etmektedir. Ardından gerçekleştirilen ilk analiz sonucunda sistem, konuşmada bir anomali bulunup bulunmadığına karar vermektedir. Anomali tespit edilmediği durumda analiz normal sonuç ile tamamlanırken, anomali tespit edilmesi durumunda detaylı hata analizi gerçekleştirilmekte, açıklama üretilmekte ve kullanıcıya ayrıntılı analiz raporu sunulmaktadır.

3.5 User Interface — navigation & mock-ups

Bu bölüm AgentLens sisteminin kullanıcı arayüzünü ve ekranlar arasındaki navigasyon akışını tanımlar. Arayüz, izlenen çok ajanlı sisteme müdahale etmeden dışarıdan çalışan bir analiz aracı için tasarlanmıştır ve bilgi mimarisi genelden özele bir hiyerarşi izler: kullanıcı önce izlenen sistemlerini görür, ardından ilgilendiği sisteme iner, o sistemin konuşmalarını listeler ve son olarak tekil bir konuşmanın detayına ulaşır.

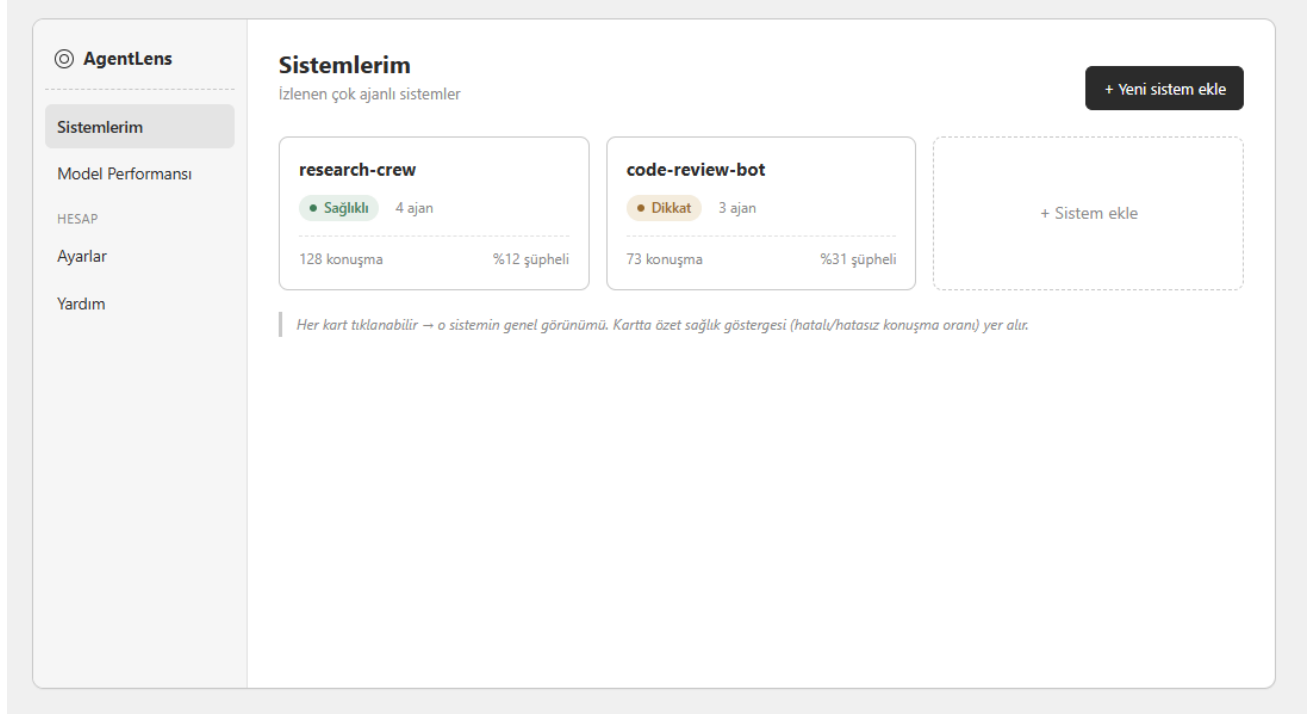
Bu bölümde sunulan ekran görüntüleri, arayüzün wireframe halini temsil etmektedir; renk paleti, tipografi ve görsel kimlik gibi tasarım kararları bilinçli olarak dışarıda bırakılmış, odak ekranların bilgi mimarisine, düzenine ve ekranlar arası akışa verilmiştir. Geliştirme sürecinde ekranların kemik yapısı (yerleşim, hiyerarşi, bileşenler ve navigasyon mantığı) bu wireframe'de gösterildiği gibi korunacak; görsel iyileştirmeler ve detay değişiklikleri ilerleyen aşamalarda uygulanacaktır.

Arayüz altı ekrandan oluşur. Sistemlerim ekranı giriş noktasıdır ve kullanıcının izlediği tüm çok ajanlı sistemleri özet sağlık göstergeleriyle birlikte listeler. Sistem Ekle ekranı üç farklı giriş yolunu tek bir yerde toplar: canlı entegrasyon için koda eklenecek bir snippet (Senaryo 1 ve 2), dışarıdan konuşma kaydı yüklemek için JSON/JSONL yükleme akışı (Senaryo 3) ve etiketli test kümesiyle performans değerlendirmesi için ayrı bir sekme (Senaryo 4). Sistem Görünümü ekranı seçilen bir sistemin sağlık istatistiğini ve filtrelenebilir konuşma listesini sunar; liste satırları kompakt tutulur (durum rozeti, tek satır sebep özeti, anomali skoru) ve ayrıntılı bilgi bir sonraki ekrana ertelenir. Aynı ekranda ikincil bir sekme olarak Ajanlar görünümü bulunur; bu görünüm, konuşma bazlı analizin tamamlayıcısı olarak ajan bazlı bir bakış açısı sağlar. Konuşma Detayı sistemin ayırt edici çıktısını görselleştiren kilit ekrandır: solda mesaj-mesaj bir konuşma akışı vurgulu hatalı adımla birlikte gösterilir, sağda ise yapılandırılmış hata raporu (hata türü, MAST FM/FC kodu, başladığı tur, ilgili ajanlar, anomali skoru, açıklama) yer alır. Son olarak Model Performansı ekranı, AgentLens dedektörünün etiketli test kümesi üzerindeki başarımlarını (F1, recall, precision, sınıflandırma doğruluğu, confusion matrix, MAST hata türü dağılımı) raporlar; bu ekran izlenen sistemin sağlığından ayrı bir kavramı temsil eder ve karıştırılmaması amaçlanır.

Ekranlar arası geçişler, use case modelindeki eylemlerle birebir örtüşür: bir sistem kartına tıklanması "View Dashboard / Logs" use case'ini, bir konuşma satırına tıklanması "Stage 2: Perform Failure Analysis" akışının kullanıcıya sunulan yüzünü, "Sistem Ekle" akışı ise "Integrate System" ve "Start Monitoring" use case'lerini görselleştirir. Ekran 3'ün üstünde yer alan uyarı banner'ı, Hata Yolu 1 (zorunlu alanı eksik log kaydı) senaryosunun arayüzdeki karşılığıdır ve FR-18 gereksinimini somutlaştırır.

Aşağıdaki ekran görüntüleri, gerçek bir çok ajanlı sistem (research-crew) örneği üzerinden arayüzün tipik kullanım akışını göstermektedir. Ekranlar arası navigasyon akışı ise bu bölümün sonundaki navigasyon akış diyagramında bir bütün olarak sunulmuştur. Arayüzün tıklanabilir prototipine aşağıdaki bağlantı üzerinden erişilebilir; ekranlar arası geçişler ve etkileşimler bu prototipte doğrudan denenebilir:

Tıklanabilir prototip: <https://agentlenswireframe.vercel.app>



Ekran-1

AgentLens

Sistemlerim

Model Performansı

Sistemlerim / Yeni sistem ekle

Yeni sistem ekle

Bir izleme yöntemi seç

Canlı entegrasyon

Kayıt yükle (JSON/JSONL)

Etiketli test kümesi

Aşağıdaki kodu kendi çok ajanlı sisteminin başına ekle. AgentLens ajan mesajlarını log üzerinden dinlemeye başlar — sistemine müdahale etmez.

```
# pip install agentlens
from agentlens import Monitor

monitor = Monitor(api_key="al_****")
monitor.attach(system="my-agents")
```

Kodu kopyala

Bağlantıyı test et →

Snippet yolu FR-1/PR-7'yi (canlı, müdahalesiz izleme); yükleme yolu NFR-8'i ve risk raporundaki bağımsız offline aracı (B-planı) tek ekranda gösterir.

Ekran-2

AgentLens

Sistemlerim

Model Performansı

Sistemlerim / research-crew

research-crew

4 ajan · son senkron 2 dk önce

Sağlıklı

128

TOPLAM KONUŞMA

113

NORMAL

15

ŞÜPHELİ

%12

ŞÜPHELİ ORANI

Konuşmalar

Ajanlar

⚠ 3 konuşmada zorunlu alan eksikliği tespit edildi (gönderen / alıcı / zaman damgası).

İncele →

Konuşmalarda ara...

Tümü

Şüpheli

Normal

Hata türü ▾

KONUŞMA	AJANLAR	DURUM	SEBEP (ÖZET)	SKOR	TARİH
#4821	Planner ↔ Coder	• Şüpheli	Adım tekrarı — aynı alt görev 3 kez	0.86	14:02
#4820	Coder ↔ Tester	• Normal	—	0.11	13:58
#4819	Planner ↔ Tester	• Şüpheli	Akıl yürütme-eylem uyumsuzluğu	0.79	13:51
#4818	Coder ↔ Planner	• Normal	—	0.09	13:40
#4817	Tester ↔ Coder	• Normal	—	0.18	13:33

Detay (hata türü/FM kodu, kanıt, tam açıklama) burada değil — satıra tıklayınca Ekran 5'te açılır. Liste özet kalır.

Ekran-3

AgentLens

Sistemlerim

Model Performansı

Sistemlerim / research-crew

research-crew

4 ajan

Konuşmalar

Ajanlar

AJAN	ROL	KONUŞMA	KARIŞTIĞI ŞÜPHELİ	EN SIK HATA
Planner	Görev ayrıştırma	96	9	Adım tekrarı
Coder	Uygulama	104	11	Akl yürütme-eylem
Tester	Doğrulama	71	4	Eksik doğrulama
Reviewer	Son onay	52	2	Erken sonlandırma

Bu görünüm, "hangi ajan sık hata üretiyor?" sorusuna yardımcı olur; ama analiz birimi hâlâ konuşma.

Ekran-4

AgentLens

Sistemlerim

Model Performansı

Sistemlerim / research-crew / #4821

Konuşma #4821

Planner ↔ Coder · 8 tur · 14:02

Şüpheli · 0.86

Planner tur 1

Adım 1

Coder tur 2

Adım 2

Planner tur 3

Adım 3

Coder tur 4

Adım 4 · tespit noktası

Adım tekrarı burada başlıyor

Planner tur 5

Adım 5

HATA RAPORU

DURUM

Şüpheli

HATA TÜRÜ

Adım Tekrarı (FM-1.3)

KATEGORİ

Sistem Tasarımı (FC1)

BAŞLADIĞI ADIM

Tur 4

İLGİLİ AJANLAR

Planner, Coder

ANOMALİ SKORU

0.86

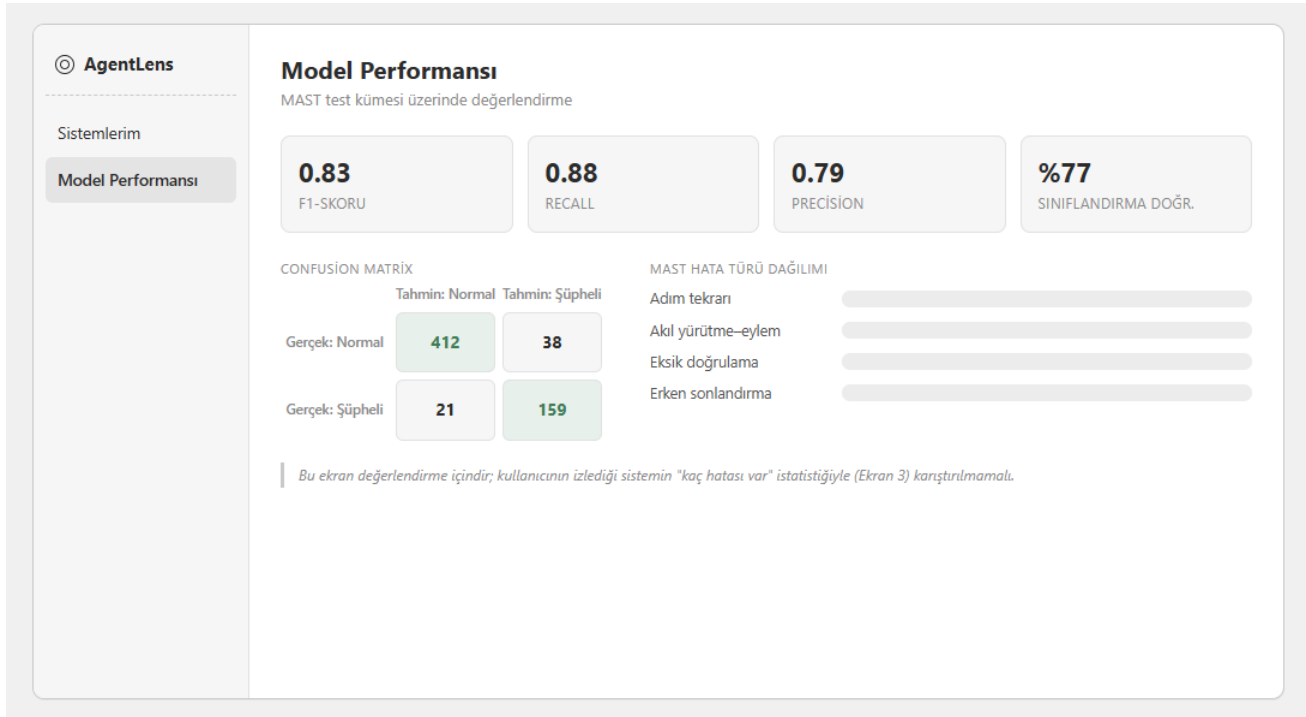
AÇIKLAMA

Coder, önceki turda tamamlanan alt görevi yeni bir görevmiş gibi tekrar ele aldı; Planner bunu durdurmadığı için aynı adım döngüye girdi.

Raporu JSON indir

"Hangi adımda" görselleştirmesi projenin en ayırt edici çıktısı — vurgulu baloncuk + rapordaki "başladığı adım" birbirini işaret eder.

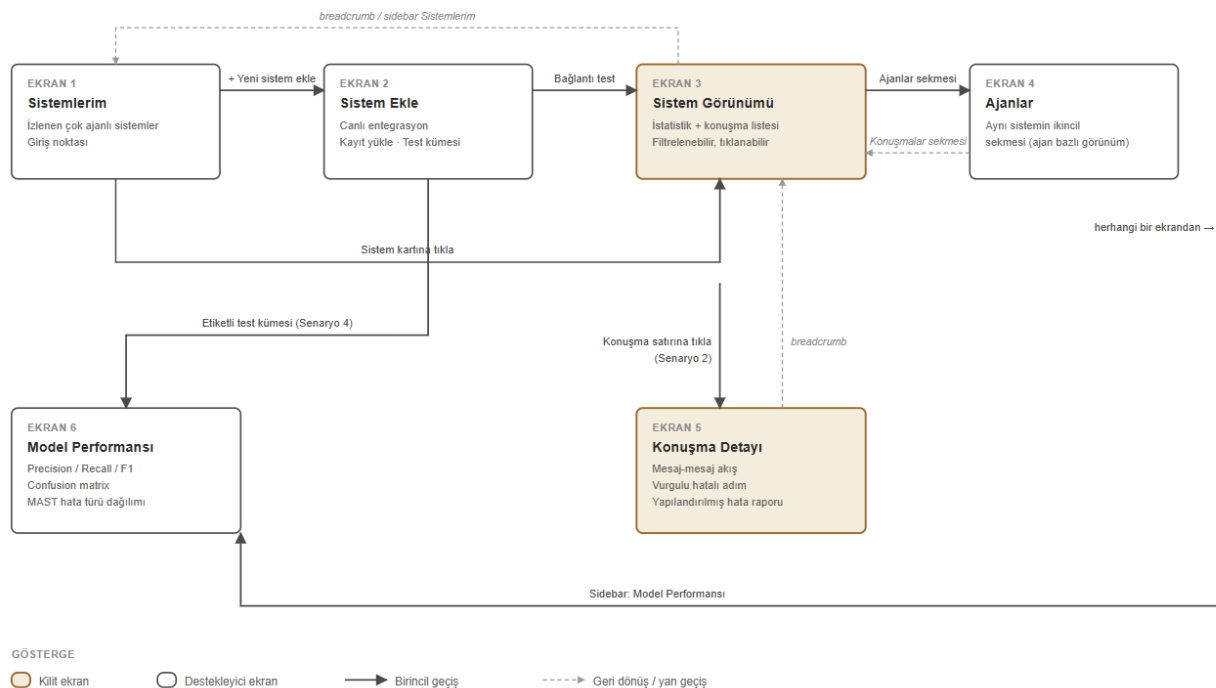
Ekran-5



Ekran-6

AgentLens — Navigation Flow

6 ekran ve aralarındaki geçişler. Düz oklar birincil akış; kesikli oklar geri dönüş / yan geçiş.



Senaryo eşleşmeleri: Senaryo 1 (Normal akış) = 1→3, listede "Normal" satır · Senaryo 2 (Hata tespiti) = 3→5 · Senaryo 3 (Offline) = 1→2→3 · Senaryo 4 (Değerlendirme) = 2→6

Navigasyon Akış Diyagramı

4. Glossary

Terim	Açıklama
Agent (Ajan)	Çok ajanlı sistem içerisinde belirli bir role, hedefe ve yeteneklere sahip bağımsız yazılım bileşeni.
Multi-Agent System (MAS)	Birden fazla ajanın ortak bir görevi gerçekleştirmek için iletişim ve koordinasyon kurduğu sistem.
AgentLens	Çok ajanlı sistemlerdeki iletişim ve koordinasyon hatalarını tespit etmek, açıklamak ve raporlamak amacıyla geliştirilen analiz sistemi.
Trace / Log	Ajanların mesajlarını, aksiyonlarını, araç çağrılarını ve çalışma zamanı bilgilerini içeren kayıtlar.
Raw Trace Data	Herhangi bir dönüşüm uygulanmadan saklanan orijinal konuşma ve sistem kayıtları.
Formatted Data	Ham kayıtların AgentLens tarafından işlenebilen standart JSON veya JSONL şemasına dönüştürülmüş hâli.
Conversation Window	Güncel mesaj ile önceki konuşma geçmişinin birleştirilmesiyle oluşturulan analiz bağlamı.
Anomaly Detection	Bir konuşma penceresinin normal veya şüpheli olarak sınıflandırılması işlemi.
Anomaly Score	Bir konuşma penceresinde anomali bulunma olasılığını veya derecesini gösteren sayısal değer.

Failure Analysis	Şüpheli konuşmaların hata türü, hata adımı, ilgili ajanlar ve hata nedeni açısından ayrıntılı olarak incelenmesi.
MAST Taxonomy	Çok ajanlı sistemlerde görülen hata ve başarısızlık türlerini kategorilere ayıran sınıflandırma yapısı.
Failure Mode	MAST taksonomisinde tanımlanan belirli bir iletişim veya koordinasyon başarısızlığı türü.
Few-Shot Prompting	Dil modeline görevi açıklamak için prompt içerisinde sınırlı sayıda örnek sunulması yöntemi.
Dynamic Retrieval	Analiz edilen girdiye anlamsal olarak en yakın örneklerin veri deposundan otomatik olarak seçilmesi işlemi.
Embedding	Metinlerin anlamsal özelliklerini sayısal vektörler şeklinde temsil eden yapı.
Structured Output	Önceden tanımlanmış alanlara ve şemaya uygun olarak üretilen JSON benzeri makine tarafından işlenebilir çıktı.
Pseudonymization	Kişisel bilgilerin, gerçek kimliği doğrudan göstermeyen takma değerlerle değiştirilmesi işlemi.
Reprocessing	Daha önce kaydedilmiş ham verinin, bir hata düzeltildikten veya sistem güncellendikten sonra yeniden işlenmesi.
Dashboard	Analiz edilen oturumların, normal akışların, hata raporlarının ve performans metriklerinin görüntülediği kullanıcı arayüzü.
Precision, Recall	Sistemin hata tespit performansını değerlendirmek için kullanılan sınıflandırma

ve F1-Score	metrikleri.
-------------	-------------

References

- [1] **Draw.io (diagrams.net)**. (2026). *Diagramming Software for System Architecture and UML Modeling*. [Online]. Available: <https://app.diagrams.net/>
- [2] **Vercel**. (2026). *Cloud Infrastructure and Deployment Documentation for Web Applications*. [Online]. Available: <https://vercel.com/docs>